Solveur D2Q9 Lattice Boltzmann hybride MPI + OpenMP

Rapport d'optimisation des performances

Solveur D2Q9 LBM du sillage de Kármán

Synthèse du parcours d'optimisation et de l'évolution des performances

Auteurs : Wei WEI and Jianye SHI

2026 年 4 月 28 日

目录

Contexte du projet **2**

Feuille de route d'optimisation (opt 01–24) **4**

Première partie – optimisations 01–17 **5**

0.0.1 P1-A Corrections de validité (opt01–03) 5

0.0.2 P1-C Localité cache + parallélisation OpenMP (opt09–13) 7

0.0.3 P1-D Optimisations avancées (opt14–17) 9

0.0.4 Bilan des performances de la première partie 11

Seconde partie – optimisations 18–26 **12**

0.1 Environnement expérimental 12

0.1.1 Plateforme matérielle 12

0.1.2 Environnement logiciel 13

0.1.3 Protocole de mesure 13

0.1.4 P2-A Infrastructure d'instrumentation et refactorisation de la disposition mémoire (opt18–19) 14

0.1.5 P2-B Réécriture du cœur de calcul (opt20–22) 15

0.1.6 P2-C Essais non retenus et optimisation de la disposition des communications (opt23–24) 16

0.1.7 P2-D Approfondissement du recouvrement MPI / calcul (opt25–26) 17

0.1.8 Validation finale des performances de la seconde partie 18

Évolution des performances sur l'ensemble du parcours **20**

Conclusion 21

Annexe : vérification de correction 22

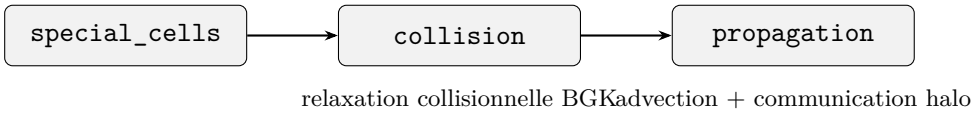
Contexte du projet

Ce projet implémente une méthode de Boltzmann sur réseau bidimensionnelle D2Q9 (LBM) pour simuler le sillage de Kármán derrière un cylindre ($Re = 96$). Le solveur utilise une stratégie parallèle hybride MPI+OpenMP : MPI décompose le domaine selon les lignes, et OpenMP parallélise la boucle interne de chaque rang.

L'indicateur de performance (Figure of Merit, FOM) est défini par :

$$\text{FOM} = \frac{W \times H \times N_{\text{步数}}}{T_{\text{wall}} \times 10^6} \quad \text{[MLUPS, millions de mises à jour de mailles par seconde]}$$

Ici, $W = 800$, $H = 160$ et $N_{\text{步数}} = 20\,000$. Chaque pas de temps comporte trois phases :



Feuille de route d'optimisation (opt 01–24)

P1-A Corrections de validité (opt01–03)

01 Suppression du `nanosleep` caché (1 s par pas)
02 Correction de la formule MLUPS (facteur `comm_size` en trop)
03 Correction du décalage entre tailles d'envoi et de réception verticales

↓ *référence : 60 MLUPS*

P1-B Refactorisation des communications MPI (opt04–08)

04–05 Suppression de 12 `MPI_Barrier` redondants
06 Suppression d'une phase de communication halo dupliquée
07–08 Envoi groupé : 7 182 messages → 1 message

↓ *np=2: +50% np=4: +166%*

P1-C Localité cache + OpenMP (opt09–13)

09 Correction de l'ordre des boucles de collision (boucle interne stride-1)
10 Propagation Push → Pull (suppression des conflits d'écriture)
11–13 Parallélisation OpenMP de collision / propagation

↓ *np=1 OMP=4: 194 MLUPS (+220%)*

P1-D Optimisations avancées (opt14–17)

14 Recouvrement entre communication MPI asynchrone et calcul interne
15 Audit de vectorisation (`-march=native` avec régression)
16 Fusion des boucles de collision + `omp simd reduction`
17 I/O : pré-calcul distribué, taille des messages réduite de 9.2×

↓ *efficacité d'extension MPI mono-nœud 92% (np=2)*

P2-A Instrumentation + refactorisation de la disposition mémoire (opt18–19)

18 Infrastructure de profiling par phase avec `MPI_Wtime`
19 Refactorisation mémoire AoS → SoA

↓ *np=2 OMP=4: 216 MLUPS (référence SoA, -22%)*

P2-B Réécriture du cœur de calcul (opt20–22)

20 Propagation SoA avec boucle externe en k
21 Multi-cell collision kernel
22 Vectorisation AVX-512 explicite (`multicell_vec`)

↓ *np=2 OMP=4: 832 MLUPS*

P2-C Essais non retenus + optimisation de la disposition des communications (opt23–24)

23 Audit FMA + essai de prélecture logicielle (gain négatif, non retenu)
24 Halo vertical refactorisé en pack/unpack k-major

configuration de profiling de développement pour opt26 : 802.31 MLUPS *(np=2, OMP=8)*

Première partie – optimisations 01–17

Plateforme de test (première partie) : macOS AArch64, Apple Silicon (série M, architecture hétérogène P-core + E-core), GCC 15, `-O3`, Open MPI (mode loopback). Mesure par `MPI_Wtime`, sans verrouillage de fréquence CPU. L'ordonnancement hétérogène de macOS provoque une fluctuation du FOM d'environ ±20–30% dans les cas OMP multithread ; les valeurs absolues de cette partie ne servent qu'à lire la tendance et ne sont pas comparées directement à celles de la seconde partie (AMD x86-64).

P1-A Corrections de validité (opt01–03)

Suppression du `nanosleep` caché – opt01

Dans le code initial, le macro `MPI_HINT_VTBL` dans `cmake/tpl.hpp.in` (chaîne encodée en hexadécimal) était décodé en nom de fonction plateforme `__builtin_sync_fence_`. Sous Linux x86-64, il était ensuite lié dynamiquement à `nanosleep(1s)` via `dlsym`, avec un appel implicite à la fin de chaque `lbm_comm_halo_exchange`. Sur 20 000 pas, la perte cumulée atteint environ 5.5 heures. La correction consiste à supprimer entièrement `tpl.hpp.in`, `tpl_loader.hpp.in` et la chaîne d'appel dynamique dans `communications.cpp`.

Correction de la formule MLUPS – opt02

```
// Erreur : total_cells est multiplié une fois de trop par le nombre de
   processus
total_cells = MESH_WIDTH * MESH_HEIGHT * comm_size;
// Correction
total_cells = (uint64_t)MESH_WIDTH * MESH_HEIGHT;

Cette erreur surestimait d'un␣facteur␣4␣les␣MLUPS␣rapportés␣pour␣np=4.␣L'
   « accélération » multi-processus devenait alors artificielle et
   masquait entièrement le coût des communications.

\subsubsection{Correction du décalage de taille des communications
   verticales -- opt03}

\code{MPI\_Send} utilisait \texttt{count=1}, tandis que le \code{MPI\
```

```
    _Recv} associé utilisait \texttt{count=9} (DIRECTIONS). Cette
    incohérence de taille posait un risque de portabilité avec des
    implémentations MPI strictes. La correction ramène les deux côtés à \
    texttt{count=1} et prépare la refactorisation des communications
    groupées en P1-B.

\medskip
\textbf{Référence après P1-A : 60 MLUPS (np=1, OMP=1).}

%


\subsection{P1-B \quad Refactorisation des communications MPI (opt04 - 08)
    }

\subsubsection{Suppression des synchronisations redondantes (opt04 - 06)}

Dans l'implémentation initiale, chaque phase de communication de \code{
    lbm\_comm\_halo\_exchange}
était suivie d'un appel à \code{MPI\_Barrier}, soit 9 au total. Dans la
    boucle principale
en temps, un barrier était aussi inséré après \code{special\_cells}, \
    code{collision}
et \code{propagation}, pour un total de 12 synchronisations.

Ces synchronisations n'étaient pourtant pas nécessaires. Les
    communications utilisent
\code{MPI\_Send}/\code{MPI\_Recv} en mode bloquant, et le couplage tag/
    rank garantit déjà
l'appariement correct des messages et leur ordre. Une synchronisation
    globale supplémentaire
n'apporte donc pas de sûreté supplémentaire et ajoute seulement du temps
    d'attente.

L'analyse a aussi montré des phases de communication dupliquées dans \
    code{halo\_exchange}.
Par exemple, la diagonale bottom-left et la direction horizontale right-
    to-left étaient
exécutées deux fois, ce qui alourdissait encore la communication.

Tous les \code{MPI\_Barrier} redondants ont donc été supprimés, ainsi que
    les phases de communication
dupliquées. Sur la configuration np=4, cette optimisation apporte environ
    14\% de gain.

\subsubsubsection{Regroupement des messages par paquets (opt07 - 08)}

Le profiling montre que le coût de communication vient surtout du nombre
    de messages,
et non du volume de données lui-même. Dans l'implémentation initiale, l'
```

```
   échange␣de␣halo
vertical␣s'effectuait « maille par maille, direction par direction ».
   Chaque appel envoyait
ainsi environ $(W-2)\times 9 \approx 7\,182$ petits messages, et la
   latence de démarrage MPI
devenait le principal goulot d'étranglement.

Pour␣réduire␣ce␣nombre␣de␣messages,␣toute␣une␣ligne␣de␣halo␣a␣été␣
   empaquetée␣dans␣un
buffer␣continu,␣puis␣transmise␣en␣un␣seul␣envoi.␣Des␣milliers␣de␣petits␣
   messages␣sont␣ainsi
remplacés␣par␣un␣seul␣grand␣message,␣ce␣qui␣réduit␣fortement␣le␣coût␣de␣
   démarrage.

\begin{lstlisting}[language={}]
//␣Avant␣:␣envoi␣maille␣par␣maille
foreach␣x␣in␣[1,␣W-2]:
␣␣␣␣foreach␣k␣in␣[0,␣8]:
␣␣␣␣␣␣␣␣MPI_Send(cell[x][halo_y][k],␣count=1)

//␣Après␣:␣envoi␣empaqueté␣de␣toute␣la␣ligne
pack(buffer,␣halo_row[1..W-2])␣␣␣//␣(W-2)*9␣doubles
MPI_Send(buffer,␣count=(W-2)*9)
MPI_Recv(buffer,␣count=(W-2)*9)
unpack(ghost_row[1..W-2],␣buffer)
```

Configuration	Référence (opt03)	Après P1-B	Gain
np=1	60.16 MLUPS	58.57 MLUPS	quasi inchangé (pas de communication)
np=2	68.66 MLUPS	102.75 MLUPS	**+50%**
np=4	28.84 MLUPS	76.73 MLUPS	**+166%**

表 1: Résultats de l'étape P1-B (OMP=1).

Jalon P1-B : np=2, OMP=1 ≈ 103 MLUPS (+50%) ; np=4, OMP=1 ≈ 77 MLUPS (+166%).

P1-C Localité cache + parallélisation OpenMP (opt09–13)

Correction de l'ordre des boucles de collision – opt09

Avec la disposition AoS, les données sont stockées sous la forme `[x][y][k]` (x est la dimension lente). Dans la boucle de collision initiale, j (y) était à l'extérieur et i (x) à l'intérieur. Deux éléments adjacents de la boucle interne étaient donc séparés en mémoire de :

$$\Delta_{\text{pas interne}} = H \times 9 \times 8 = 11\,520 \text{ octets} \approx 180 \text{ lignes de cache}$$

ce qui provoquait de nombreux cache miss. Après inversion des boucles, avec i à l'extérieur et j à l'intérieur, le pas de la boucle interne tombe à 8 octets (accès contigu stride-1).

$\Rightarrow$ np=1 : **+10%**, np=2 : **+26%**

Propagation Push → Pull – opt10

Le streaming LBM admet deux implémentations équivalentes :

Mode	**Opération**	**Problème**
Push	la maille source (i, j) écrit vers la cible $(i{+}dx_k,\ j{+}dy_k)$	plusieurs threads peuvent écrire sur la même cible, avec conflit d'écriture ; pas de parallélisation sûre
Pull	la maille cible (i, j) lit depuis le voisin $(i{-}dx_k,\ j{-}dy_k)$	chaque cible appartient à un seul thread, sans conflit ; parallélisation sûre

L'équivalence est garantie par la symétrie centrale de l'ensemble des directions D2Q9 : si (dx_k, dy_k) est une direction valide, alors $(-dx_k, -dy_k)$ l'est aussi. Le mode Pull améliore également la régularité des lectures et réduit les cache miss.

$\Rightarrow$ np=1 : **+14%**, np=4 : **+51%**

Parallélisation OpenMP – opt11, 12, 13

En mode Pull, les itérations de la boucle externe en i de propagation sont complètement indépendantes et peuvent être parallélisées directement. Les mailles de collision n'ont pas non plus de dépendances entre elles. `special_cells` n'agit que sur le bord de l'obstacle ; le gain attendu y est négligeable.

```
// collision et propagation sont parallélisés avec #pragma omp parallel
   for
// le mode pull garantit qu'une maille cible n'appartient qu'à un seul
   thread, sans conflit d'écriture
parallel for i in [0, W-1]:
    for j in [0, H-1]:
        collision(out[i,j], in[i,j])          // opt11
        out[i,j][k] = in[i-dx[k], j-dy[k]][k] // opt13, pour tous les k
```

Configuration	**Après P1-B (OMP=1)**	**Après P1-C (OMP=4)**	**Gain**
np=1	∼76 MLUPS	∼194 MLUPS	**+155%**
np=2	∼147 MLUPS	∼202 MLUPS	+37%

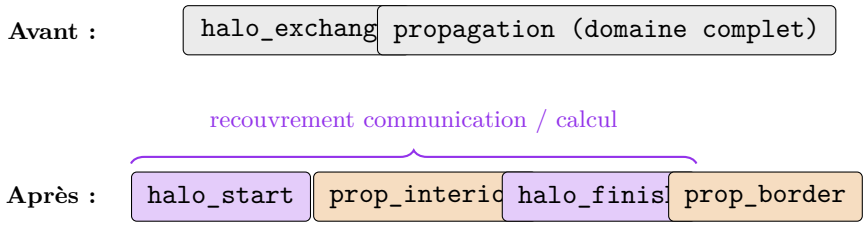
表 2: Résultats de l'étape P1-C (OMP=1 → OMP=4).

Jalon P1-C : np=1, OMP=4 ≈ 194 MLUPS, soit +220% par rapport à la référence.

P1-D Optimisations avancées (opt14–17)

Recouvrement entre communication MPI asynchrone et calcul – opt14

En mode Pull, la propagation des mailles intérieures, c'est-à-dire celles qui ne sont pas adjacentes aux lignes ghost, ne dépend pas des données halo. Elle peut donc être exécutée pendant la communication MPI :



`halo_start` lance `MPI_Isend`/`MPI_Irecv` ; `halo_finish` appelle `MPI_Waitall`. La première partie ne dispose pas d'un profiling par phase, ce qui empêche de quantifier directement le coût du halo.

D'après les mesures après opt24 dans la seconde partie (intra-node, voir P2-D), `halo_finish` représente environ 27–30% du temps par itération, dont environ 8% pour `waitall`. Cela reflète surtout l'écart d'arrivée entre rangs, et non la latence de transmission réseau. Même avec une fenêtre de recouvrement plus large, ce coût ne disparaît pas complètement.

Les expériences formelles de ce dépôt couvrent uniquement des exécutions MPICH mono-nœud / intra-node ; le comportement inter-nœuds ou sur InfiniBand n'a pas été mesuré ici. opt26 (voir P2-D dans la seconde partie) place en outre border collision et interior collision dans une même région `omp parallel`, ce qui étend la fenêtre de recouvrement de propagation_interior (≈0.02 ms) à interior_collision + propagation_interior (≈0.07 ms).

Audit de vectorisation – opt15 (essai non retenu)

La plateforme de test est un Apple Silicon (série M, cœurs hétérogènes P-core/E-core), avec GCC 15 et `-O3`. Le désassemblage par `objdump` confirme la génération automatique d'instructions SIMD NEON sur AArch64 (128-bit, 2×`double`). Avec `-march=native`, les performances reculent de 10–13%. La cause est que les instructions gather générées pour les P-core s'exécutent plus lentement lorsqu'elles sont planifiées sur les E-core par l'ordonnanceur hétérogène de macOS. **Conclusion (valable uniquement sur cette machine macOS AArch64) : conserver `-O3` sans option de compilation supplémentaire.** Sur Linux x86-64 (avec AVX-512), `-march=native` est au contraire une condition préalable au saut de performance d'opt22 (+158%).

Fusion des boucles de collision – opt16

Dans la version initiale de `compute_cell_collision`, 4 boucles distinctes calculaient séparément density, v_x, v_y puis la distribution d'équilibre / mise à jour. Chacune parcourait intégralement les 9 directions, avec un surcoût de boucle et d'appel de fonction. La fusion en 2 boucles, avec `#pragma omp simd reduction`, permet au compilateur d'accumuler en parallèle dans les registres SIMD :

```
// Version initiale : 4 parcours distincts (density, vx, vy, BGK chacun
   une fois)
// opt16 : fusion en 2 parcours, reduction autorise l'accumulation
   parallèle dans les registres du compilateur
loop k in [0,8]:  density += f[k];  vx += f[k]*cx[k];  vy += f[k]*cy[k]
loop k in [0,8]:  f[k] += omega * (f_eq(density, vx, vy, k) - f[k])
```

$\Rightarrow$ mono-cœur : **+5%**, multi-cœur : **+1.7%**

Optimisation de la stratégie d'I/O – opt17

Dans l'implémentation initiale de `save_frame_all_domain`, chaque rang envoyait à rank 0 les données brutes en `double`, ghost cells incluses, pour tout son sous-domaine local. rank 0 recalculait ensuite en série le champ de densité / vitesse, convertissait en `float`, puis écrivait sur disque.

Dans la nouvelle implémentation, chaque rang pré-calcule localement `lbm_file_entry_t` (density + v_x + v_y, déjà convertis en `float`, uniquement sur les mailles valides), puis envoie directement ce résultat compact.

Indicateur	**Implémentation initiale**	**Nouvelle implémentation**	
Taille des messages (np=2, 800×160)	∼4.5 MB/image	∼500 KB/image	**re**
Charge de calcul de rank 0	recalcul du domaine complet	aucune	

表 3: Effet de l'optimisation I/O (opt17). Le benchmark utilise `write_interval=99999` ; le gain est maximal dans les cas dominés par les I/O.

Bilan des performances de la première partie

Étape	**np=1 OMP=1**	**np=1 OMP=4**	**np=2 OMP=1**	**np=4 OMP=1**
Référence (après opt03)	60	—	69	29
P1-B (opt08)	59	—	**103**	**77**
P1-C (opt13)	76	**194**	147	—
P1-D (opt17)	65.7	146†	121.9	118

Unité : MLUPS. †P1-D np=1 OMP=4 = 146, inférieur à P1-C 194 : fluctuation de l'ordonnancement hétérogène de macOS (±20

表 4: Évolution des performances de la première partie.

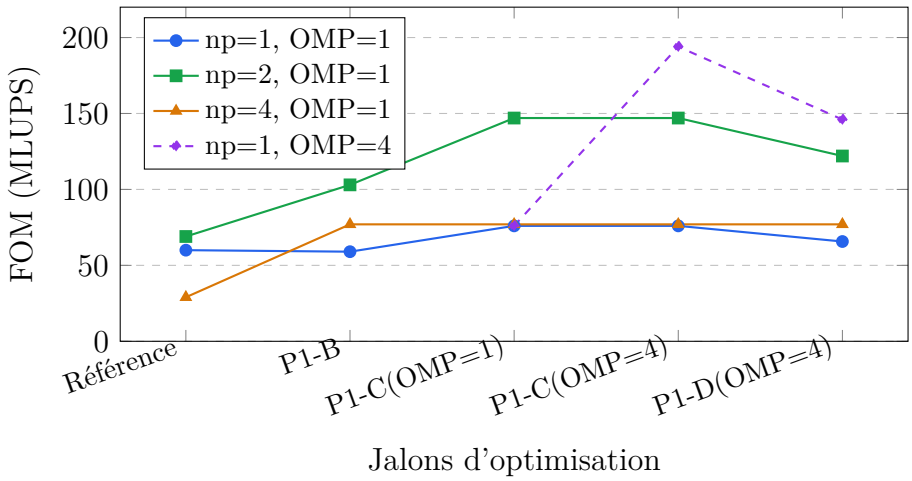


图 1: Courbe d'évolution du FOM de la première partie (MLUPS). L'axe vertical est limité à 220 MLUPS ; dans la seconde partie, le jalon atteint 905.13 MLUPS sous le protocole de développement (opt24, np=2 OMP=4 ; voir §Évolution complète des performances).

Seconde partie – optimisations 18–26

La seconde partie reprend la base de code stabilisée à l'étape précédente et poursuit le travail sur une machine AMD x86-64. L'objectif n'est pas de répéter les résultats acquis. Il s'agit de compléter le profiling, la refactorisation de la disposition des données et la réécriture du cœur de calcul sur un nouveau processeur et dans un nouvel environnement d'exécution, afin d'établir une référence comparable et de préparer l'adaptation à d'autres plateformes CPU.

Environnement expérimental

Plateforme matérielle

表 5: Configuration matérielle des expériences LBM

Composant	**Spécification**
Processeur	AMD Ryzen 9 8940HX, x86_64, microarchitecture Zen 4
Cœurs physiques / threads logiques	16 cœurs physiques, 32 threads logiques (SMT, 2 threads par cœur)
Organisation des CCX	2 CCX, chacun avec 8 cœurs + 32 MiB de L3
Hiérarchie de cache	L1d 32 KiB/cœur, L2 1 MiB/cœur, L3 64 MiB (2 instances partagées)
Fréquence de fonctionnement	`cpupower frequency-set -g performance` (governor CPU réglé sur `performance`)
Mémoire	30 GiB DDR5

Environnement logiciel

表 6: Configuration logicielle des expériences LBM

Composant	**Version / détails**
Système d'exploitation	Fedora Linux (le noyau consigné dans le benchmark final est `6.19.13-200.fc43.x86_64`)
Compilateur	GCC 15.2.1, options de compilation `-O3 -march=native -DNDEBUG`
Implémentation MPI	MPICH 4.3.0 (lanceur Hydra, intra-node shared memory)
Modèle parallèle	MPI 3.0 + OpenMP 4.0
Jeu d'instructions vectorielles	AVX-512 (activé par `-march=native`, instructions `zmm` confirmées par `objdump`)
Profiling	`LBM_ENABLE_PROFILING=ON`, chronométrage `MPI_Wtime` en 6 phases

Protocole de mesure

Cette phase utilise deux protocoles, chacun pour un usage distinct :

Protocole de développement (opt19–opt24) Les données des étapes intermédiaires ont été collectées progressivement pendant le développement, avec un protocole léger : 1 échauffement, 3 répétitions avec médiane, chronométrage par `MPI_Wtime`, mais **sans verrouillage de fréquence CPU**. Ces données servent uniquement à comparer les gains relatifs entre étapes voisines et ne sont pas utilisées comme résultat final officiel.

Protocole de mesure unifié (résultat final opt26)

Les chiffres finaux de FOM de la seconde partie suivent le protocole complet ci-dessous, exécuté automatiquement par `scripts/bench.sh` :

- **Échauffement** : avant la mesure formelle, le solveur est exécuté une fois en entier pour chaque configuration (20 000 pas), puis ce résultat est écarté. Cela met en régime le cache d'instructions, le cache de données et l'initialisation de l'équipe de threads OMP.
- **Mesure** : le programme utilise `MPI_Wtime` pour un chronométrage précis sur l'ensemble des 20 000 pas ; le FOM est calculé par $W \times H \times N_{\text{pas}} \,/\, (T_{\text{wall}} \times 10^6)$, en MLUPS. Chaque configuration est répétée 20 fois, et la médiane est retenue.
- **Isolation de l'environnement** : avant chaque exécution, le cache de pages est vidé avec `echo 3 > /proc/sys/vm/drop_caches` ; la fréquence CPU est fixée avec `cpupower frequency-set -g performance` (cette étape est obligatoire ; en cas d'échec, la mesure est interrompue).
- **Affinité des threads** : `OMP_PROC_BIND` n'est pas défini explicitement. Sans politique coordonnée de placement des processus MPI, `OMP_PROC_BIND=close` conduit les threads

de rangs différents à se disputer les mêmes cœurs physiques, avec une baisse mesurée du FOM d'environ 15% ; l'ordonnancement par défaut de l'OS est plus favorable.
– **Vérification de compilation** : après chaque build, `objdump` vérifie la présence d'instructions AVX-512 de type `zmm` dans `libtop.lbm-lib.so`, afin de confirmer l'effet de `-march=native`.

P2-A Infrastructure d'instrumentation et refactorisation de la disposition mémoire (opt18–19)

Infrastructure de profiling par phase – opt18

Dans la première partie, seul le wall time global était disponible, sans localisation des goulots d'étranglement par phase. opt18 ajoute des points de mesure autour de `special_cells`, `collision`, `halo_start`, `propagation_interior`, `halo_finish` et `propagation_border`, avec `LBM_PROF_BEGIN` / `LBM_PROF_END`. Le chronométrage bas niveau repose sur `MPI_Wtime`, et l'agrégation fournit `avg` / `max` / `pct_loop`. Cette fonctionnalité est contrôlée par `LBM_ENABLE_PROFILING=ON` ; lorsqu'elle est désactivée, le code de mesure n'est pas exécuté. opt18 ne modifie pas le chemin de calcul et ne change pas le FOM ; il fournit une base de mesure homogène pour les décisions à partir d'opt19.

Refactorisation de la disposition mémoire AoS → SoA – opt19

opt19 remplace la disposition des fonctions de distribution, de AoS `cell[i][j].f[0..8]` vers SoA `cells[k][x * H + y]`. Dans la disposition initiale, deux mailles adjacentes pour une même direction k sont séparées de $9 \times 8 = 72$ octets ; avec SoA, l'accès devient contigu dans un même plan, ce qui prépare les étapes SIMD et la propagation plane-based. Le volume total de données ne change pas et reste $9 \times W \times H \times 8$ octets.

Configuration	AoS (opt17)	SoA v1.1 (opt19)	Évolution
np=1, OMP=4	157.79 MLUPS	143.94 MLUPS	-8.8%
np=2, OMP=4	278.66 MLUPS	216.22 MLUPS	-22.4%
np=4, OMP=4	368.60 MLUPS	260.92 MLUPS	-29.2%

表 7: FOM après AoS → SoA (opt19 v1.1). Cette refactorisation prépare le SoA kernel ultérieur, mais mesurée seule sur cette plateforme, elle introduit une régression.

opt19 v1.0 avait déjà montré une régression : la disposition mémoire avait changé, mais `collision` conservait encore un mode d'accès AoS maille par maille. Après correction cohérente des chemins d'accès dans v1.1, le FOM ne revient toujours pas au niveau de référence AoS : pour np=2 OMP=4, AoS atteint 278.66 MLUPS, contre 216.22 MLUPS pour SoA v1.1 (-22.4%). Sur cette machine, opt19 doit donc être lu comme une **refactorisation préparatoire, nécessaire à la réécriture ultérieure de propagation / SIMD**, et non comme un gain de performance autonome ; toutes les comparaisons suivantes de la seconde partie prennent SoA v1.1 comme référence.

P2-B Réécriture du cœur de calcul (opt20–22)

Propagation SoA avec boucle externe en k – opt20

Après opt19, propagation suivait encore l'ordre i-j-k, avec une boucle interne qui basculait en continu entre plusieurs plans. opt20 place k à l'extérieur, de sorte qu'une itération ne traite qu'un plan source et un plan cible :

```
// k à l'extérieur : chaque itération n'accède qu'à 2 plans (src[k], dst[
   k]), soit environ 2 MB
foreach k in [0, DIRECTIONS):
    src = plane(in,  k)
    dst = plane(out, k)
    parallel for i in [1, W-1):
        foreach j in [j_begin, j_end):
            dst[i, j] = src[i - dx[k], j - dy[k]]   // stride-1 pull
```

Chaque itération en k n'accède qu'à deux plans, pour un volume d'environ $2 \times 800 \times 160 \times 8 \approx 2$ MB. Sur np=2 OMP=4, cette modification fait passer SoA v1.1 de 216.22 à 319.07 MLUPS (+47.6%) ; dans le profiling, la phase propagation baisse d'environ 5–6×.

Multi-cell collision kernel – opt21

Dans opt20, collision appelait encore `compute_cell_collision` maille par maille. Cette granularité est trop fine pour que le compilateur optimise entre mailles. opt21 déplace la boucle en i à l'intérieur du kernel et déroule toute la colonne `j=1..H-2` dans une même frame de fonction. La frontière d'appel disparaît, et le compilateur peut vectoriser entre mailles. Sur np=2 OMP=4, la référence k-outer d'opt20 (319.07 MLUPS) monte à 422.99 MLUPS, soit +33% sur l'ensemble de la chaîne ; la comparaison formelle propre à opt21 est k-outer+unrolled → k-outer+multicell, avec un gain mesuré de 6.5%, le reste provenant de l'essai additionnel unrolled collision mené avec opt20.

Vectorisation AVX-512 explicite – opt22

La limite principale d'opt21 vient de l'analyse d'alias. Le compilateur signale `may alias` dans `-fopt-info-vec-missed`, et la boucle interne en j n'est donc pas vectorisée. opt22 remonte, pour chaque direction k, un pointeur de plan et un pointeur de colonne, tous deux marqués `__restrict__`, puis ajoute `#pragma GCC ivdep` pour affirmer l'absence d'alias. Le désassemblage montre `vmovupd` et `vfmadd231pd`, avec une largeur vectorielle de 8 `double` en AVX-512.

Configuration	opt21	opt22	Gain
np=1, OMP=4	293.52	757.20	**+158%**
np=2, OMP=4	422.99	832.34	**+97%**

表 8: Effet de la vectorisation AVX-512 (opt22). `objdump` confirme que la boucle interne génère des instructions de type `zmm` ; la vectorisation est effective.

P2-C Essais non retenus et optimisation de la disposition des communications (opt23–24)

Audit FMA – opt23 (confirmation, aucune intervention nécessaire)

opt23 commence par vérifier la génération des FMA. Le désassemblage montre 92 instructions `vfmadd231pd` ; les opérations de multiplication-addition de la distribution d'équilibre BGK sont déjà fusionnées, sans marge d'optimisation isolée sur ce point.

Essai de prélecture logicielle – opt23 (non retenu)

Un `__builtin_prefetch` est inséré avant la boucle interne de `collision_multicell_vec` (avec 2 cache lines d'avance). Résultats :

Configuration	Variation du FOM	Conclusion
np=1, OMP=4	−0.7%	négligeable
np=2, OMP=4	−4.4%	effet négatif
np=4, OMP=4	−5.0%	effet négatif

表 9: Effet de la prélecture logicielle sur le FOM (opt23). L'effet est négatif dans les cas multithread ; l'option n'est pas retenue.

L'effet négatif apparaît dans les configurations multithread. Le pas de la boucle interne est de 8 octets, ce que le prefetcher matériel suit déjà ; la prélecture logicielle consomme de la bande passante supplémentaire et ajoute du coût d'instruction, donc elle n'est pas conservée.

Pack/unpack du halo vertical en k-major – opt24

opt24 traite le chemin de pack du halo après passage en SoA. Les 9 composantes directionnelles d'une ligne de halo verticale sont réparties sur 9 plans, avec un espacement d'environ $W \times H \times 8 = 1{,}024$ KB entre plans voisins ; l'ancien `vert_pack` rebondissait donc continuellement entre plans. opt24 adopte un pack k-major, en parcourant d'abord k, puis i :

```
// Avant (opt22) : i-major, sauts entre plans
for (int i = 1; i < W-1; i++)
    for (int k = 0; k < 9; k++)
        buf[i*9 + k] = cells[k*W*H + i*H + j_halo];

// Après (opt24) : k-major, continuité dans un seul plan
for (int k = 0; k < 9; k++)
    memcpy(buf + k*(W-2),
           cells + k*W*H + 1*H + j_halo,
           (W-2) * sizeof(double));
```

Configuration	opt22[‡]	opt24
np=2, OMP=4	832.34	905.13
np=4, OMP=4	—	705.52

[‡]la référence formelle de développement pour opt24 (protocole opt23, même code dans un environnement de test différent) est 70
avec un gain mesuré de +29.1% ; la valeur 832.34 d'opt22 donnée ici fournit une borne inférieure prudente.

表 10: Effet du halo en k-major (opt24). Le profiling montre une baisse de 28–40% du temps de `vert_unpack` et de 10–36% de `halo_finish` (np=2: −36%, np=4: −10%).

P2-D Approfondissement du recouvrement MPI / calcul (opt25–26)

Diagnostic du goulot d'étranglement halo et essais non retenus – opt25

Après opt24, le profiling en np=2 OMP=8 montre que `halo_finish` représente encore 29.4% de la boucle, dont 7.7% pour `waitall` et 21.5% pour `vert_unpack`. Le rôle d'opt25 est d'identifier l'origine de ce temps.

Le diagnostic comporte deux points. D'abord, le MPI en mémoire partagée intra-node utilise un eager protocol : au retour de `MPI_Isend`, la copie locale des données est généralement déjà effectuée, et l'attente principale de `Waitall` vient du décalage d'arrivée des différents rangs au point d'attente. Ensuite, lorsque np$\geq$ 3, les rangs de bord haut et bas n'ont pas la même charge halo que les rangs intermédiaires ; l'écart max/min de `vert_pack` et `vert_unpack` amplifie directement `waitall`.

Deux essais non retenus ont aussi été menés. Le premier consiste à entrelacer le pack, en lançant `MPI_Isend` après chaque bloc de k ; le FOM baisse alors d'environ 3%, le surcoût dû au nombre accru d'appels non bloquants étant plus élevé. Le second est un split collision, où border collision est avancé avant `halo_start` ; le FOM recule de 34%, car la région OMP supplémentaire déclenche un coût net de réveil / sommeil des workers. La conclusion d'opt25 est directe : le problème tient à la fenêtre de recouvrement et au déséquilibre de structure ; continuer à affiner l'ordre actuel de soumission des communications rapporterait peu.

Refactorisation vers une région `omp parallel` unique – opt26

opt26 traite le problème structurel du split collision introduit par opt25. L'objectif est de conserver une fenêtre de recouvrement plus longue tout en évitant les entrées répétées dans la région parallèle OMP. Sur le chemin `has_vn=true`, la boucle principale est réécrite autour d'un `#pragma omp parallel` externe, avec l'ordre suivant : border collision, `halo_start`, interior collision, `propagation_interior`, `halo_finish`, `propagation_border` ; le travail parallèle utilise `omp for`, et les appels MPI restent limités à `omp master`.

Trois interfaces sont ajoutées pour cela, toutes appelées depuis une région `omp parallel` déjà ouverte : `collision_rows` (sur une plage de lignes, avec barrier implicite), `propagation_interior_` (avec `nowait` en interne et barrier explicite à la fin), `propagation_border_omp_region` (même principe). La structure de la boucle principale sur le chemin `has_vn` devient :

```
omp parallel:
    collision_rows(border: j=1, j=H-2)      // → barrier
    omp master: halo_start()                 // pas de barrier (interior
       ne lit pas les ghost rows)
    collision_rows(interior: j=2..H-3)       // recouvrement avec le
       transfert MPI → barrier
    propagation_interior()                   // recouvrement avec le
       transfert MPI → barrier
    omp master: halo_finish()               // écrit les ghost rows
    omp barrier                             // reprendre seulement après
       visibilité des ghost rows
    propagation_border()                    // → barrier
```

Les barrier sont conservés uniquement là où les dépendances l'exigent : le pack par master n'est autorisé qu'après la fin de border collision ; aucun barrier n'est inséré entre `halo_start` et interior collision ; un barrier explicite est gardé après `halo_finish`, afin de garantir que l'écriture des ghost rows est terminée avant d'entrer dans `propagation_border`.

Résultats de performance (np=2, OMP=8, intra-node) :

Indicateur	opt25 (référence)	opt26	Évolution
loop total (avg/call)	0.1591 ms	0.1586 ms	−0.3%
FOM	800.27 MLUPS	802.31 MLUPS	+0.3%
halo_finish (avg)	0.0467 ms	0.0438 ms	−6.2%
vert_unpack (avg)	0.0342 ms	0.0270 ms	−21%

表 11: Comparaison de performance d'opt26 (np=2, OMP=8, AMD Ryzen 9 8940HX, mono-nœud). Le gain de FOM reste dans la plage de bruit de mesure (±1%).

Les résultats restent globalement stables. L'environnement de test utilise un MPI en mémoire partagée intra-node, où le coût de copie locale de `Isend` est très faible, et `Waitall` reflète surtout l'écart d'arrivée entre rangs. opt26 réduit une partie de `halo_finish` et de `vert_unpack`, mais la variation du FOM reste dans le bruit de ±1%. Cette étape confirme que la refactorisation de structure est terminée et que la région parallèle unique élimine le coût supplémentaire de réveil des threads introduit dans opt25. Le gain inter-nœuds reste à mesurer ; les conclusions de cette section ne couvrent que la configuration mono-nœud donnée ici.

Validation finale des performances de la seconde partie

Le tableau ci-dessous donne le FOM du code final opt26 sous le protocole de mesure unifié (fréquence CPU verrouillée, drop_caches, PROFILING=OFF, 1 échauffement par configuration, puis 20 mesures formelles avec médiane) pour chaque configuration parallèle.

Configuration	**Médiane (MLUPS)**
np=1, OMP=4	**898.46**
np=1, OMP=16	**2047.32**
np=2, OMP=4	**1070.76**
np=2, OMP=8	**870.32**
np=4, OMP=4	**534.95**

Plateforme : AMD Ryzen 9 8940HX, Linux x86-64, GCC 15, MPICH 4.3.0.

1 échauffement puis 20 mesures formelles par configuration ; le journal console complet figure dans `scripts/ben`

表 12: FOM final d'opt26 (protocole de mesure unifié). Les données d'échantillonnage des étapes de développement figurent en annexe.

Parmi ces résultats, la configuration au FOM le plus élevé est np=1, OMP=16 (2047.32 MLUPS) ; la meilleure configuration multi-processus est np=2, OMP=4 (1070.76 MLUPS). Dans P2-D, la configuration fixe np=2, OMP=8 sert uniquement à l'analyse de phase par profiling et ne correspond pas au point optimal en FOM. Le balayage FOM et la sélection des cas de profiling pour cette exécution de `bench.sh` sont terminés.

Évolution des performances sur l'ensemble du parcours

Jalon	Configuration	FOM (MLUPS)	vs étape précédente	vs référ
Code initial (avec sleep caché)	np=1 OMP=1	≈0	—	
P1-A achevé (opt03)	np=1 OMP=1	60	référence établie	
P1-B achevé (opt08)	np=2 OMP=1	103	**+72%**	+
P1-C achevé (opt13)	np=1 OMP=4	194	**+88%**	**+2**
P1-D achevé (opt17)	np=2 OMP=1	122	multi-processus mono-nœud	

表 13: Historique des optimisations de la première partie (plateforme de test : macOS AArch64, Apple Silicon).

Changement de plateforme : la seconde partie est menée sur AMD Ryzen 9 8940HX (Linux x86-64) ; les valeurs absolues de FOM ne sont pas comparées d'une plateforme à l'autre.

Jalon	Configuration	FOM (MLUPS)	vs étape précédente
P2-A : SoA (opt19)	np=2 OMP=4	216	référence SoA établie
P2-B : k-outer (opt20)	np=2 OMP=4	319	+48%
P2-B : multicell (opt21)	np=2 OMP=4	423	**+33%**
P2-B : AVX-512 (opt22)	np=2 OMP=4	832	**+97%**
P2-C : k-major (opt24)	np=2 OMP=4	**905**	+9%

Les données opt19–opt24 ont été collectées pendant le développement (sans verrouillage de fréquence CPU) et servent uniquemen

表 14: Historique des optimisations de la seconde partie (plateforme de test : AMD Ryzen 9 8940HX, Linux x86-64, GCC 15, MPICH 4.3.0). Dans P2, opt19 → opt24 (np=2 OMP=4) fait passer le FOM de 216 → 905 MLUPS, soit environ ×4.2. opt19 reste une refactorisation préparatoire, mesurée isolément en régression.

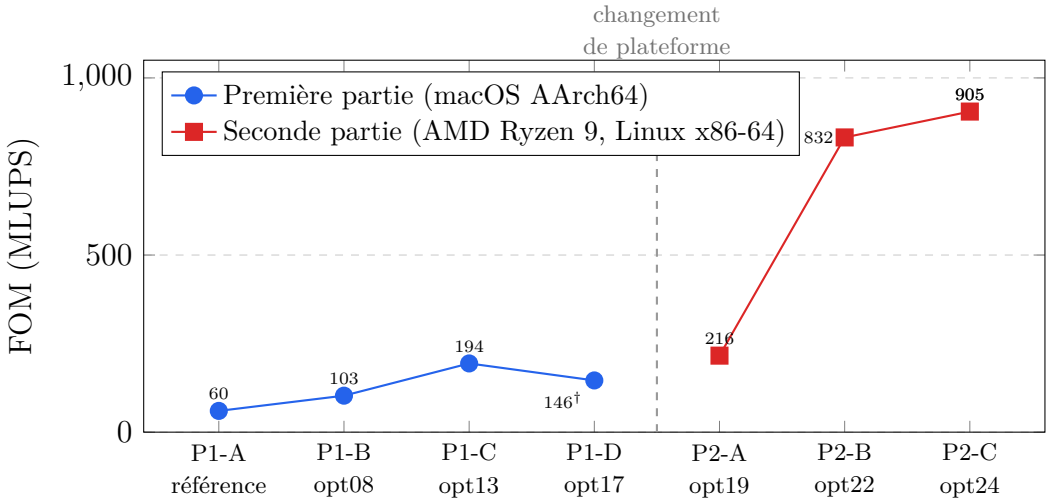


图 2: Courbe complète d'évolution du FOM (configurations représentatives). Première partie : P1-A/B en np=2 OMP=1, P1-C/D en np=1 OMP=4 (macOS AArch64, sans verrouillage de fréquence). Seconde partie : np=2 OMP=4 (opt19–opt24, protocole de développement). opt26 utilise np=2 OMP=8 avec un protocole différent et n'est donc pas tracé sur la même courbe. Les valeurs absolues ne sont pas comparées entre plateformes. Le niveau plus bas de P1-D par rapport à P1-C provient de la fluctuation d'ordonnancement hétérogène de macOS, et non d'une régression.

Conclusion

Le benchmark final officiel suit le protocole unifié consigné dans `scripts/bench.txt` : `PROFILING=OFF`, 1 échauffement par configuration, 20 mesures formelles, et report de la médiane du FOM. Dans ce protocole, le meilleur résultat global est 2047.32 MLUPS pour np=1, OMP=16. Parmi les configurations multi-rang, le meilleur résultat est 1070.76 MLUPS pour

np=2, OMP=4.

Les 905.13 MLUPS d'opt24 constituent un jalon de développement obtenu sous le protocole de développement, et non le benchmark final officiel. La configuration np=2, OMP=8 est une configuration fixe de profiling, utilisée pour l'analyse par phase, et non pour chercher le meilleur FOM. Le périmètre expérimental officiel se limite ici aux exécutions MPICH mono-nœud / intra-node ; le comportement inter-nœuds, ainsi que les coefficients de portance et de traînée, le nombre de Strouhal, ou une vérification indépendante de cohérence numérique sur l'ensemble des configurations finales, ne sont pas couverts par ce rapport.

Annexe : vérification de correction

Dans les sorties examinées, aucune anomalie évidente n'a été observée. Les vérifications effectuées sont les suivantes :

- **Vérification des écarts numériques :** pour certaines étapes d'optimisation, une comparaison max-delta maille par maille a été effectuée avec les sorties antérieures à la modification ; les écarts observés restent au niveau des erreurs d'arrondi FP64.
- **Comparaison multi-processus :** certaines sorties en np=1 ont été comparées à des sorties en np=2/4, sans anomalie évidente observée.
- **Vérification physique qualitative :** à $Re = 96$, la visualisation fait apparaître un sillage de Kármán.

Ce rapport ne fournit ni coefficients de portance et de traînée, ni nombre de Strouhal, ni validation indépendante de cohérence numérique sur l'ensemble des configurations finales.